Hands-on AI for Accessibility Testing — Trainee Cheatsheet

Companion to the video. Every prompt in the session, ready to copy-paste into ChatGPT, Claude, or Gemini. Plus sample HTML to practice on and a verification checklist for the SBA.

> **How to use this sheet:** Open the AI of your choice in one tab and this cheatsheet in another. Copy a prompt, paste it, replace the placeholders, run it. Verify the output against the checklist at the end. That's the whole loop.

Setup checklist (one-time, ~10 minutes)

For Modules 1 and 2 you just need a browser and an AI chat tab. For Module 3 (Playwright tests) you'll need a bit more:

```
# Node.js 18+ required
node --version

# Create a project folder
mkdir a11y-tests && cd a11y-tests
npm init -y

# Install Playwright + axe-core integration
npm install -D @playwright/test @axe-core/playwright
npx playwright install

# You're ready. Save AI-generated test scripts as tests/<name>.spec.ts
# Run with: npx playwright test
```

Part 1 — The foundation: WCAG context block

Paste this at the top of every audit/fix prompt. It dramatically reduces hallucinations.

```
REFERENCE — WCAG 2.2 A & AA (core criteria):

Perceivable
```

```
1.1.1 Non-text Content — Provide text alternatives for non-text content.
1.3.1 Info and Relationships — Information, structure, and relationships conveyed
1.3.5 Identify Input Purpose — Programmatically determinable purpose for form fie
1.4.3 Contrast (Minimum) — Text contrast ratio of at least 4.5:1 (3:1 for large t
1.4.10 Reflow — Content can be presented at 320px width without 2D scrolling.
1.4.11 Non-text Contrast — UI components and graphical objects have 3:1 contrast.

Operable
2.1.1 Keyboard — All functionality is operable through a keyboard interface.
2.1.2 No Keyboard Trap — Keyboard focus can be moved away from any focused compon
2.4.3 Focus Order — Focusable components receive focus in an order that preserves
2.4.6 Headings and Labels — Headings and labels describe topic or purpose.
2.4.7 Focus Visible — Keyboard focus indicator is visible.
2.5.3 Label in Name — The accessible name contains the visible label text.

Understandable
3.2.1 On Focus — No context change when a component receives focus.
3.2.2 On Input — No context change when a user enters data into a field.
3.3.1 Error Identification — Errors are identified and described in text.
3.3.2 Labels or Instructions — Labels or instructions are provided when content r

Robust
4.1.2 Name, Role, Value — For all UI components, name and role can be programmati
4.1.3 Status Messages — Status messages can be programmatically determined throug
```

Module 1 — Audit HTML, write defect tickets

Prompt 1: Audit HTML for WCAG 2.2 violations

```
You are a senior accessibility engineer with deep WCAG 2.2 expertise.

[PASTE THE WCAG CONTEXT BLOCK FROM ABOVE HERE]

TASK: Audit the HTML below against WCAG 2.2 Level A and AA. Find every violation

FORMAT — for each violation:
1. Issue Title — one line, descriptive
2. WCAG Criterion — number + name + level (e.g., "1.4.3 Contrast (Minimum) — Leve
3. Affected Element — CSS selector or short HTML snippet
4. User Impact — what screen reader / keyboard / low-vision users experience
5. Recommended Fix — corrected code, not just description
6. Severity — Critical / Major / Minor
```

```
CONSTRAINTS:
- Group issues by WCAG criterion
- Prefer semantic HTML over ARIA (First Rule of ARIA)
- Don't repeat generic advice — be specific to this snippet
- If something looks suspicious but you're unsure, flag it as "needs manual verif

--- HTML ---
<paste your HTML snippet here>
```

Prompt 2: Convert violations into JIRA-ready defects

```
You are a senior accessibility engineer writing JIRA-ready defect tickets for an

TASK: Convert each accessibility violation below into a complete defect ticket us

REQUIRED FORMAT (use exactly):

Title: [concise, descriptive — what's broken]
Priority: [P1-Critical / P2-High / P3-Medium]
WCAG Criterion: [number + name + level]
Tested Build Version: {{BUILD_LINK}}
Affected Page/Component: [where it occurs]
Steps to Reproduce:
  1. [step]
  2. [step]
  3. [step]
Expected Result: [what should happen]
Actual Result: [what happens now]
User Impact: [who's affected, how]
Suggested Fix: [code-level if applicable]
Screenshot Required: Yes/No

CONSTRAINTS:
- Always include Priority (never skip)
- Always use the full {{BUILD_LINK}} placeholder — I will replace it
- One ticket per violation (don't combine)
- Group similar violations only if root cause is identical

--- VIOLATIONS ---
<paste Prompt 1 output OR raw axe-core JSON here>
```

Sample HTML to practice on — broken signup form

Use this with Prompt 1. You should see 5–6 violations.

```
<!-- Practice snippet: a broken signup form -->
<div class="form">
  <div class="heading">Sign up for our newsletter</div>
  <input type="text" placeholder="Your email">
  <input type="text" placeholder="Your name">
  <div onclick="submit()" style="color:#999;background:#eee">
    Subscribe
  </div>
  <img src="badge.png">
</div>
```

Expected violations include: missing labels (1.3.1, 3.3.2), non-semantic heading (1.3.1), non-button submit (2.1.1, 4.1.2), low contrast (1.4.3), missing alt text (1.1.1).

Module 2 — Generate ARIA/HTML fixes

Prompt: Fix a broken component

````
You are a senior frontend accessibility engineer. I will paste an HTML/JSX snippe

TASK:
1. Identify what's wrong (cite the WCAG criterion)
2. Output the fixed HTML/JSX — full snippet, ready to paste
3. Explain why your fix works (1-2 sentences max)
4. If you added ARIA, justify why native HTML wasn't sufficient

CONSTRAINTS — First Rule of ARIA applies:
- Prefer semantic HTML over ARIA every time
- No ARIA = better than bad ARIA
- Don't introduce new accessibility issues
- Preserve the existing visual design — change semantics and structure only
- Keep the same framework (HTML stays HTML, JSX stays JSX)
- Don't add features that weren't there before

OUTPUT FORMAT:

## Issue
[what's broken + WCAG ref]

## Fixed Code
```html
<corrected snippet>
````
````

Why this works

[1-2 sentences]

ARIA justification (if used)

[why native HTML wasn't enough]

— BROKEN SNIPPET — <paste broken HTML/JSX here>

```
### Sample HTML — custom dropdown that needs fixing

```html
<div class="dropdown">
  <div class="trigger" onclick="toggle()">Select country</div>
  <div class="options" style="display:none">
    <div onclick="pick('IN')">India</div>
    <div onclick="pick('US')">USA</div>
    <div onclick="pick('UK')">UK</div>
  </div>
</div>
```

The good AI response will propose a native `<select>` first (simplest fix), and only fall back to the ARIA combobox pattern if you need fancy styling.

Sample HTML — icon-only button

```
<button onclick="deleteItem()">
  <svg width="16" height="16"><path d="M6 19c0 1.1.9 2 2 2h8c1.1 0 2-.9 2-2V7H6v1
</button>
```

Expected fix: add `aria-label="Delete item"`. (This is one of the rare cases where ARIA is the right answer — there's no visible text, so `aria-label` is necessary to give the button an accessible name.)

Module 3 — Playwright + axe-core test scripts

Prompt 1: Base test script

```
You are a senior QA automation engineer specializing in accessibility testing wit
```
```

```
TASK: Write a complete, runnable Playwright test file (TypeScript) that:
1. Navigates to a given URL
2. Waits for the page to be fully loaded (networkidle)
3. Runs axe-core accessibility analysis against WCAG 2.2 Level A and AA
4. Fails the test if any violations are found
5. Logs each violation with: id, impact, description, affected nodes (with select
6. Saves the full report as JSON to ./reports/{page-name}-a11y-{timestamp}.json

REQUIREMENTS:
- Use @axe-core/playwright AxeBuilder
- Use TypeScript with proper types
- Include necessary imports
- Add a brief comment block at the top with: purpose, how to run, dependencies
- Use the withTags() method to specify wcag2a, wcag2aa, wcag21a, wcag21aa, wcag22
- Group output by impact level (critical → minor)
- Make selectors copyable (so devs can find the element fast)

INPUTS:
- URL to test: {{URL}}
- Page name for the report file: {{PAGE_NAME}}

Output: the complete .spec.ts file. No commentary outside the code block.
```

To run what the AI generates:

```
# Save the AI output as tests/audit.spec.ts
# Make sure the reports directory exists
mkdir -p reports

# Run it
npx playwright test tests/audit.spec.ts

# Open the HTML report
npx playwright show-report
```

Prompt 2: Filter by specific WCAG criteria

```
You are a senior QA automation engineer.

TASK: Extend the Playwright + axe-core test below to run ONLY the rules tagged wi

CRITERIA TO TEST:
{{CRITERIA_LIST}}
(Example: ["wcag143", "wcag246", "wcag412"])
```

```
(Format follows axe-core tag naming.)

CONSTRAINTS:
- Use withTags() with the exact criterion tags
- Don't drop the impact-level grouping in output
- Add a clear comment at the top stating which criteria this test covers
- If a criterion has no matching axe rule, log a note: "No automated rule for X.X

--- EXISTING TEST SCRIPT ---
<paste the base script from Prompt 1 here>
```

Axe-core tag reference (the most common ones):

WCAG Criterion	axe-core tag
1.1.1 Non-text Content	`wcag111`
1.3.1 Info and Relationships	`wcag131`
1.4.3 Contrast (Minimum)	`wcag143`
1.4.11 Non-text Contrast	`wcag1411`
2.1.1 Keyboard	`wcag211`
2.4.6 Headings and Labels	`wcag246`
2.4.7 Focus Visible	`wcag247`
3.3.2 Labels or Instructions	`wcag332`
4.1.2 Name, Role, Value	`wcag412`
4.1.3 Status Messages	`wcag413`

Pattern: strip the dots from the criterion number, prefix with `wcag`. So 1.4.3 → `wcag143`.

Prompt 3: Generate a custom axe-core rule

```
You are an accessibility test engineer writing a custom rule for axe-core to exte

RULE TO IMPLEMENT:
{{RULE_DESCRIPTION}}
(Example: "Every <button> element that has only an icon (no visible text node) mu
```

```
DELIVERABLES:
1. The check function (vanilla JS, runs in the browser context)
2. The rule definition in axe-core's configure() format
3. A short HTML test snippet that WOULD trigger this rule
4. A short HTML test snippet that WOULD NOT trigger it (negative test)
5. Playwright registration code showing how to load the rule before analyze()

CONSTRAINTS:
- Tag the rule with the matching WCAG criterion (e.g., "wcag412")
- Provide help and helpUrl fields pointing to WCAG docs
- Use a unique rule ID with a client prefix (e.g., "acb-icon-button-name")
- Severity: critical / serious / moderate / minor — pick appropriately
- Output as four clearly labeled code blocks

Output: just the code. No explanatory prose between blocks beyond brief comments.
```

Custom rule ideas to practice with

Try generating each of these as a custom axe-core rule:

1. **Icon-only buttons must have an accessible name** (aria-label or aria-labelledby)
2. **All form fields must have an associated `<label>`** (linked via `for` / `id`)
3. **Decorative images must have `alt=""`, not omit the alt attribute**
4. **Headings must not skip levels** (no `<h1>` directly to `<h3>`)
5. **Modal dialogs must have `role="dialog"` and `aria-labelledby`** pointing to the title

---

Bonus: chaining prompts

Once you're comfortable, chain prompts together for whole workflows:

Workflow: Page → filed tickets

1. Run Module 1 Prompt 1 (audit) on the HTML
2. Take the output, run Module 1 Prompt 2 (defect writer) on it
3. Take any high-priority violation, run Module 2 prompt to get the fix
4. Run Module 3 Prompt 1 on the page URL, save the test, run it to confirm

In about 15 minutes you've gone from raw page → audited → ticketed → fixed → automated regression test. That's the loop.

Verification checklist — before every submission

For the SBA, run through these on every artifact the AI gives you:

For audit output

- Every WCAG criterion number exists (cross-check against W3C WCAG 2.2 Quick Reference)
- Every violation cites a real element (CSS selector or HTML snippet, not “the button”)
- Severity is appropriate (Critical = blocks users completely, Major = significant impact, Minor = inconvenience)
- No generic advice (“improve contrast” without telling you to what ratio)
- User impact is specific to the disability community affected

For defect tickets

- Priority field present (P1/P2/P3)
- Tested Build Version uses {{BUILD_LINK}} placeholder
- Steps to reproduce are numbered and actionable
- Suggested fix is code, not description
- One ticket per violation (no combining unrelated issues)

For ARIA/HTML fixes

- Native HTML used wherever possible
- ARIA only present where native HTML can’t do the job
- Fix actually works when you paste it into a browser
- axe DevTools confirms no new violations introduced
- Visual design is preserved

For Playwright test scripts

- `npx playwright test` runs without errors
- Report file is written to disk

- Violations are grouped by impact
- WCAG tags are specified via `withTags()`
- If using a filtered test, the right criteria are targeted

For custom axe rules

- Rule ID has a client prefix (e.g., `acb-`)
- Tagged with the matching WCAG criterion
- Has `help` and `helpUrl` fields
- Positive test snippet actually triggers the rule
- Negative test snippet actually passes the rule
- Severity level is appropriate

Common pitfalls — quick reference

Pitfall	Fix
AI cites a non-existent WCAG criterion	Always paste the WCAG context block; verify numbers against W3C
AI sprinkles ARIA everywhere	Add "First Rule of ARIA — prefer semantic HTML" constraint
AI flags false-positive contrast issues	Verify contrast with browser DevTools or axe DevTools
AI gives generic recommendations	Add "be specific to this snippet, no generic advice"
Generated Playwright code doesn't run	Paste the error back: "fix this error: <paste>" — usually works in 1-2 rounds
AI suggests harmful fixes (e.g., `role="presentation"` on data tables)	Trust your training; look up anything that surprises you

---

Resources to bookmark

- **W3C WCAG 2.2 Quick Reference** — for criterion lookup and verification
- **Deque University** — free courses on a11y fundamentals
- **MDN ARIA Authoring Practices Guide** — when you need a specific ARIA pattern
- **axe-core rule list** — for the full list of automated rules and tags
- **WebAIM Contrast Checker** — for contrast verification

What the SBA will likely test

Based on the learning outcomes, expect to be asked to demonstrate:

1. Running an audit prompt on a snippet you're given
2. Producing one defect ticket in the team's standard format
3. Generating a fix for one issue, with First Rule of ARIA applied
4. Walking through a generated Playwright + axe-core test script
5. Identifying at least one thing the AI got wrong, and correcting it

The fifth item is the one that separates a pass from a strong pass. Prepare for it specifically — when reviewing AI output during practice, force yourself to find one thing wrong before moving on.

Good luck. The techniques in this sheet are the same ones used in production by the A11y team — start practicing tonight, and by your SBA, this should feel routine.